

DigiBank Dynamic Security Analysis

Runtime security validation of the DigiBank API.

1. Why DAST is needed

Static analysis examines source code and configuration before execution. Dynamic analysis complements it by interacting with the application while it is running. This matters because a system can compile, pass unit tests, and still expose an API route without authentication, return excessive technical detail, accept invalid input, or handle sessions incorrectly.

DAST checks what an external client can actually observe, not only what the source code appears to intend.

The sequence is: establish runtime behavior, identify security weaknesses, apply targeted remediation, and replay the same scenarios to demonstrate the result.

2. Application under test

The external client does not call each business service directly. The API Gateway is the security boundary and the single entry point for the business API. Behind it, the customer, account, transaction, compliance, and notification services run as separate Compose services.

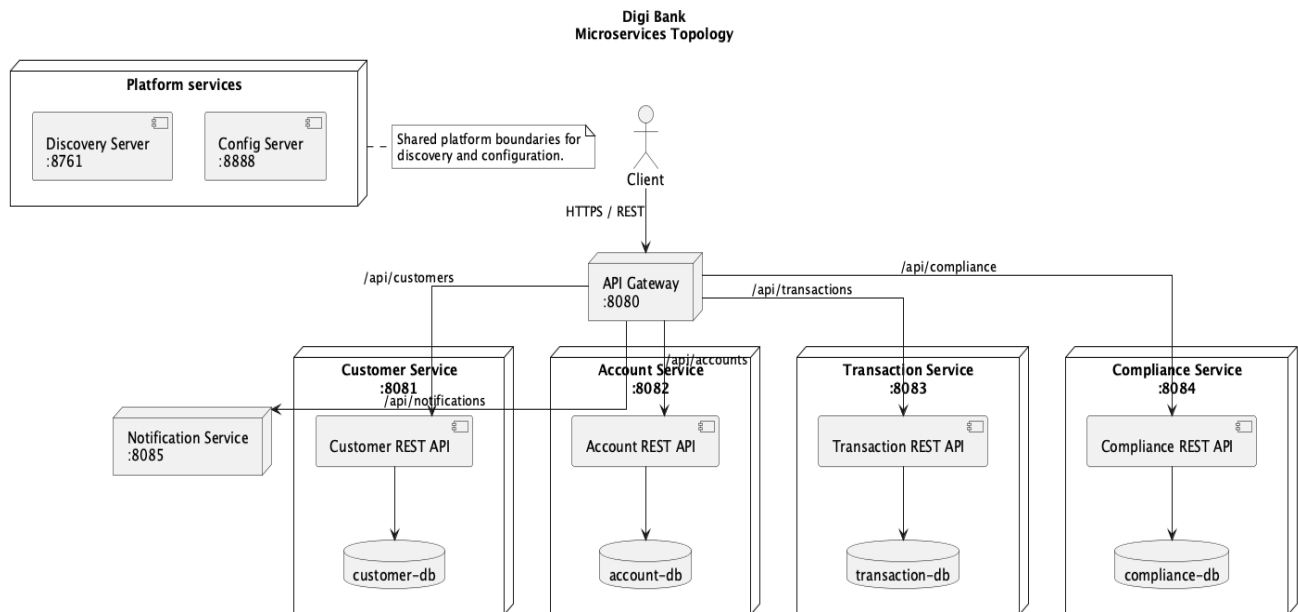


Figure 1. Runtime topology under test: gateway, business services, platform services, Compose host and persistence boundaries.

Surface	What is tested
Gateway	Authentication, authorization, headers, cache behavior, error responses and route protection.
Business APIs	Customer, account, transaction and compliance operations through gateway routing.
Runtime behavior	Malformed requests, invalid values, missing resources, session behavior and information disclosure.
Delivery path	Build, Compose startup, readiness, Newman replay and ZAP scanning.

3. Reproducible test environment

The environment is deliberately reproducible. Maven verifies the implementation, Docker Compose starts the complete topology, smoke checks verify the endpoints, and stack validation confirms that the expected containers are running before DAST begins.

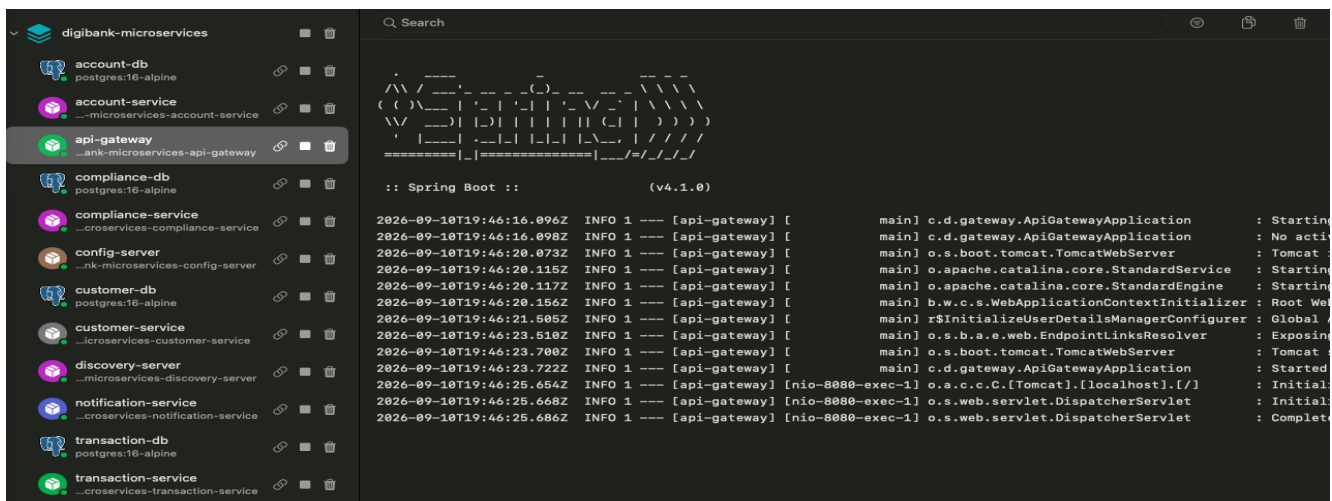
```
make clean
make build
make image-build
make up
make smoke
make validate-stack
make newman-scan
make zap-scan
```

3.1 Starting the application

The Compose startup creates the complete local runtime. The screenshot shows the command used to start the services and the resulting container state. This establishes the execution context before any security request is sent.

```
→ digibank-core git:(main) ✕ make up
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices up
docker compose -f docker-compose.yml up -d
[+] up 12/12
✓ Container digibank-microservices-config-server-1      Running      0.00s
✓ Container digibank-microservices-transaction-db-1    Healthy      1.25s
✓ Container digibank-microservices-compliance-db-1     Healthy      0.79s
✓ Container digibank-microservices-compliance-service-1 Running      0.00s
✓ Container digibank-microservices-account-db-1        Healthy      0.79s
✓ Container digibank-microservices-account-service-1   Running      0.00s
✓ Container digibank-microservices-notification-service-1 Running      0.00s
✓ Container digibank-microservices-transaction-service-1 Running      0.00s
✓ Container digibank-microservices-discovery-server-1  Running      0.00s
✓ Container digibank-microservices-customer-db-1       Healthy      0.79s
✓ Container digibank-microservices-customer-service-1  Running      0.00s
✓ Container digibank-microservices-api-gateway-1       Running      0.00s
```

The container view confirms that the gateway, business services, databases, discovery server and config server are present in the same runtime topology.



3.2 Smoke validation

Smoke validation checks the public health endpoint, gateway business routes, direct internal service endpoints, discovery registry and config server. It is not the security campaign; it proves that the application is alive and that later DAST failures can be interpreted as application behavior rather than startup failure.

```
→ digibank-core git:(main) ✗ make smoke
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices smoke
./scripts/smoke-test.sh
Checking gateway-health http://localhost:8080/actuator/health
Checking customer-api http://localhost:8080/api/customers
Checking account-api http://localhost:8080/api/accounts
Checking transaction-api http://localhost:8080/api/transactions
Checking compliance-api http://localhost:8080/api/compliance
Checking customer-service http://localhost:8081/api/customers
Checking account-service http://localhost:8082/api/accounts
Checking transaction-service http://localhost:8083/api/transactions
Checking compliance-service http://localhost:8084/api/compliance
Checking notification-service http://localhost:8085/actuator/health
Checking discovery-registry http://localhost:8761/registry
Checking config-server http://localhost:8888/config/customer-service
Microservice smoke checks passed.
```

The successful result is the gate before Newman and ZAP.

4. Baseline and remediation decision

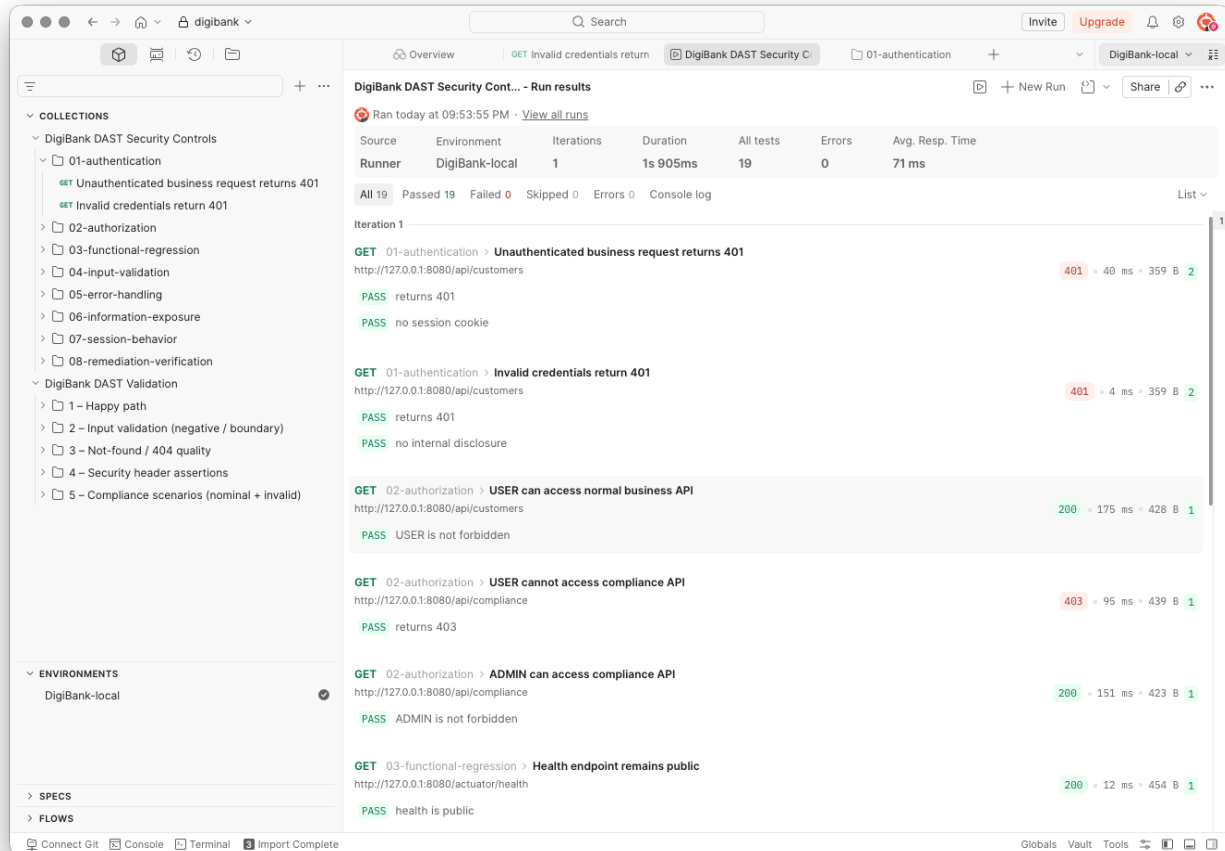
The baseline establishes the behavior before the gateway security boundary was introduced. The important observations were unauthenticated business routes, no gateway role distinction, incomplete response hardening, and no explicit cache policy for sensitive API responses.

The remediation uses stateless HTTP Basic authentication at the gateway because it is simple to reproduce in the workshop and preserves the existing service topology. It is not presented as the production identity architecture.

Request condition	Expected result after remediation
No credentials or invalid credentials	401 Unauthorized
Valid USER on normal banking API	Request is permitted
Valid USER on compliance API	403 Forbidden
Valid ADMIN on compliance API	Request is permitted
Authenticated request	No server session or session cookie

4.1 Authentication and authorization in Postman

The security collection makes the access rules visible through actual requests. The run shows unauthenticated requests returning 401, normal USER access returning 200, USER access to compliance returning 403, and ADMIN access to compliance returning 200.



The request results correspond directly to the gateway rules: authentication establishes identity, and authorization decides whether that identity may use the route.

4.2 Validation, errors and information exposure

The security collection also checks malformed JSON and missing resources. These cases return controlled 4xx responses without stack traces or internal implementation details. The same collection verifies the response headers on the health endpoint.

The screenshot displays the DigiBank DAST Security Controls interface. The left sidebar shows a tree view of collections and environments. The main panel shows the results of a test run titled 'DigiBank DAST Security Controls - Run results'. The test was run today at 09:53:55 PM. The summary table shows 19 tests passed, 0 failed, 0 skipped, and 0 errors, with an average response time of 71 ms. The test results are listed below the summary table, showing various test cases and their outcomes.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DigiBank-local	1	1s 905ms	19	0	71 ms

All 19 Passed 19 Failed 0 Skipped 0 Errors 0 Console log

Test results:

- POST 04-input-validation > Malformed JSON is rejected without disclosure
http://127.0.0.1:8080/api/customers
400 - 92 ms - 447 B 2
PASS returns a client error
PASS no stack trace
- GET 05-error-handling > Missing resource uses controlled response
http://127.0.0.1:8080/api/customers/999999999
404 - 110 ms - 496 B 2
PASS returns not found or downstream unavailable
PASS no internal details
- GET 06-information-exposure > Health response has security headers
http://127.0.0.1:8080/actuator/health
200 - 4 ms - 454 B 3
PASS content type protection
PASS frame protection
PASS cross-origin policy
- GET 07-session-behavior > Authenticated request is stateless
http://127.0.0.1:8080/api/customers
200 - 158 ms - 428 B 1
PASS no session cookie is created
- GET 08-remediation-verification > Authentication remains enforced after remediation
http://127.0.0.1:8080/api/customers
401 - 4 ms - 359 B 2
PASS no internal details

The run shows the validation and error-handling assertions passing, including the controlled 400 and 404 responses.

4.3 Headers and stateless behavior

The headers check verifies browser and response protections. The session check verifies that an authenticated request does not create a server session or return a session cookie.

The top screenshot displays the 'Health response has security headers' control. It is a GET request to `{{baseUrl}}/actuator/health`. The response is a 200 OK status with a 36 ms response time and 454 B body size. The response body is a JSON object:

```
1 {
2   "groups": [
3     "liveness",
4     "readiness"
5   ],
6   "status": "UP"
7 }
```

The bottom screenshot displays the 'Authenticated request is stateless' control. It is a GET request to `{{baseUrl}}/api/customers`. The response is a 200 OK status with a 201 ms response time and 519 B body size. The response body is a JSON array of two customer objects:

```
1 [
2   {
3     "id": 1,
4     "firstName": "DAST",
5     "lastName": "Customer",
6     "email": "dast.1789069699173@example.com"
7   },
8   {
9     "id": 2,
10    "firstName": "DAST",
11    "lastName": "Customer",
12    "email": "dast.1789073822431@example.com"
13  }
14 ]
```

These screenshots show the two related controls: hardened responses and stateless authentication.

5. Runtime checks and scan results

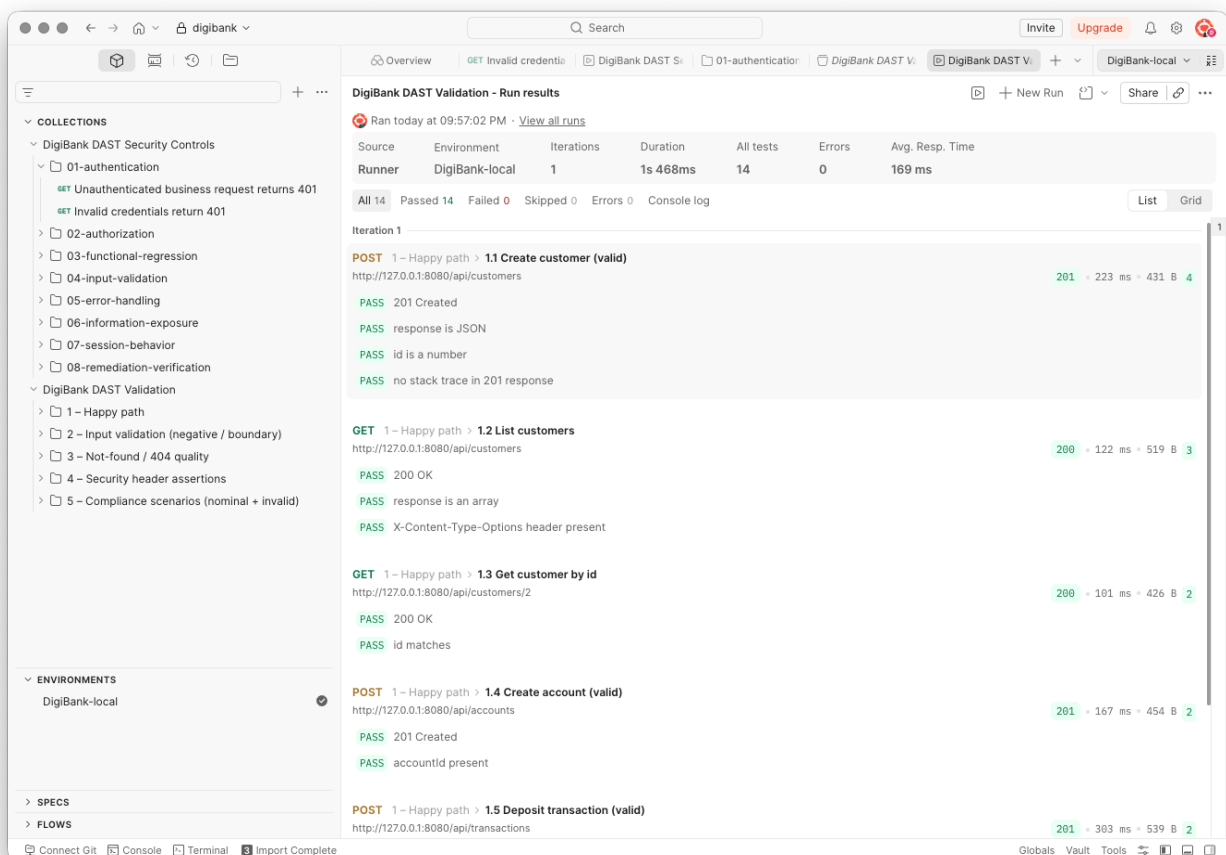
Newman expresses the expected security behavior as repeatable assertions. The security collection is divided into authentication, authorization, functional regression, input validation, error handling, information exposure, session behavior, and remediation verification.

Campaign	Result
Functional Newman collection	24 requests, 74 assertions, 0 failures.
Security Newman collection	12 requests, 19 assertions, 0 failures.
OWASP ZAP baseline	0 High, 1 Medium, 0 Low, 1 Informational.

The remaining Medium ZAP alert is the expected local warning that HTTP Basic credentials are sent over plain HTTP. It is acceptable only in this local educational environment and requires TLS in production. The Informational observation concerns public static-resource cacheability; authenticated API responses receive Cache-Control: no-store, private.

5.1 Functional regression in Postman

The functional collection checks the banking flows after the security changes: customer creation and retrieval, account creation, transactions and compliance operations. This confirms that the gateway protection did not break normal application behavior.



The collection run shows the banking requests and their passing assertions in the local environment.

5.2 Newman security replay

Newman repeats the same security collection without relying on the Postman interface. The summary is the reproducible command-line result used locally and in CI.

```
08-remediation-verification
Authentication remains enforced after remediation
GET http://127.0.0.1:8080/api/customers [401 Unauthorized, 359B, 6ms]
✓ authentication remains enforced
✓ no internal disclosure
```

	executed	failed
iterations	1	0
requests	12	0
test-scripts	12	0
prerequest-scripts	0	0
assertions	19	0

```
total run duration: 1074ms
total data received: 1.24kB (approx)
average response time: 64ms [min: 4ms, max: 200ms, s.d.: 59ms]
[INFO] Functional report: /Users/adorsys123/dev/digibank-core/dast/reports/202609
5/functional-newman-report.json
```

The security replay completed 12 requests and 19 assertions with zero failures.

Directory listing for /

- [functional-newman-report.json](#)
- [security-newman-report.json](#)

The report directory contains separate functional and security JSON results.

5.3 OWASP ZAP scan

ZAP scans the externally visible gateway surface and complements the targeted Postman assertions. The scan checks the running application for observable response, header and authentication weaknesses that are not covered by a small set of targeted requests.

The scan completed with no High or Low alerts. The remaining findings are reviewed below and are interpreted against the local test environment rather than treated as an unexplained score.

5.4 ZAP alert summary

The ZAP summary reports zero High alerts, one Medium alert, zero Low alerts and one Informational alert. The remaining Medium alert is the local plain-HTTP warning associated with HTTP Basic authentication. It is recorded and explained in the report.

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 2xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 4xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://127.0.0.1:8080	Count of total endpoints	3

6. CI validation

The CI workflow repeats the same validation chain after the application is built. Each step has a clear purpose: start the runtime, prove readiness, verify the exposed routes, replay the security assertions, and scan the gateway surface.

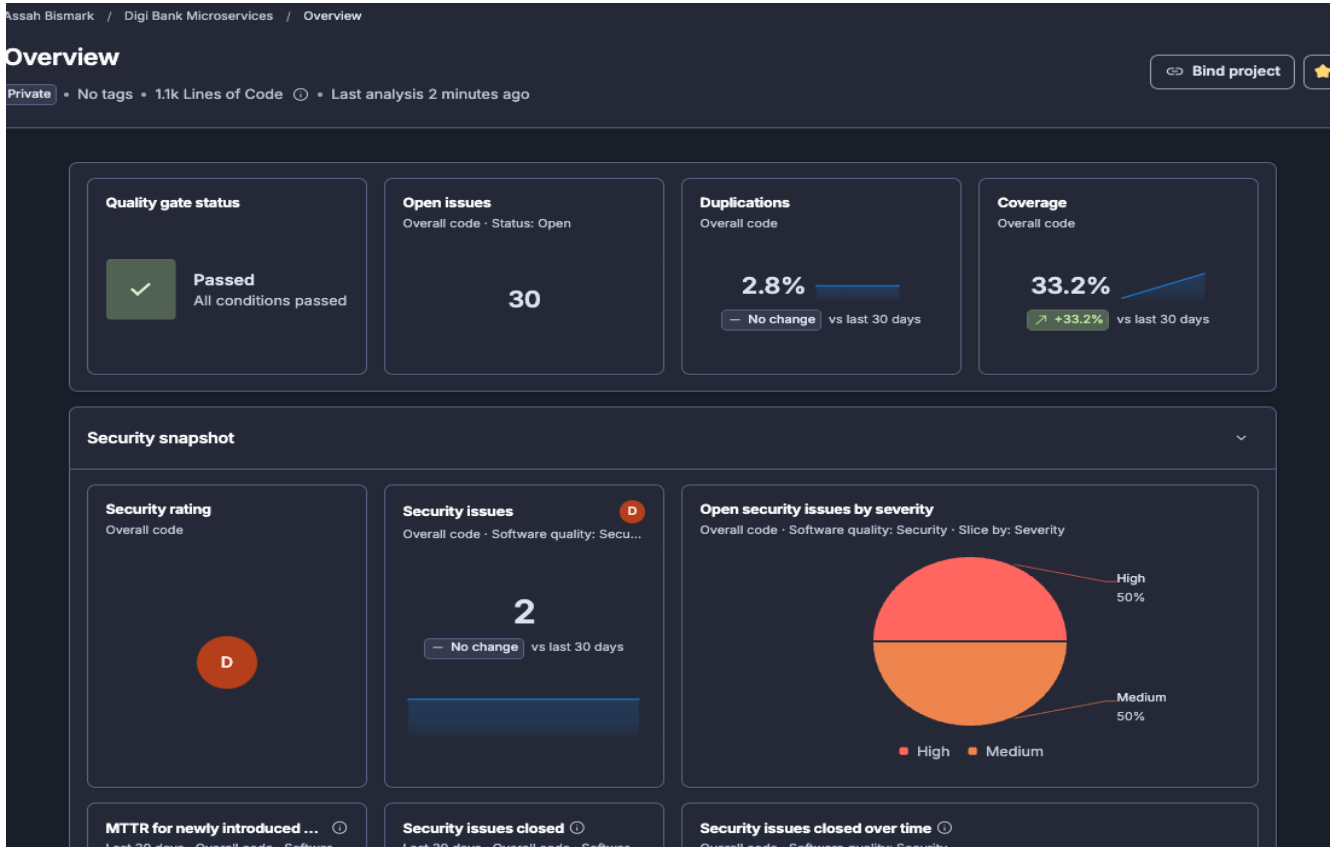
Pipeline step	Purpose
Build and tests	Compile the services and run the automated test suite.
Compose and readiness	Start the complete runtime and wait for the required endpoints.
Smoke validation	Confirm gateway, business APIs and platform services respond.
Newman	Run the versioned functional and security request assertions.
ZAP	Run an independent baseline scan against the running gateway.
Report upload	Keep Newman and ZAP results available when a check fails.

Newman and ZAP remain separate checks: Newman verifies expected security behavior, while ZAP performs broader automated observations of the exposed HTTP surface.

7. Static security analysis

Workshop 2 moves security checks earlier in the delivery lifecycle. Instead of waiting until the application is running, the source code, dependencies and test strength are inspected during the build. SonarCloud provides code and security analysis, JaCoCo supplies test coverage data, and PITest measures whether tests detect deliberately introduced defects.

The analysis is applied to the current Java 21 Spring Boot microservices implementation. It does not replace the Newman and ZAP runtime checks; it complements them.



SonarCloud quality gate passed. The dashboard reports 33.2% overall coverage, 2.8% duplication and 30 open issues.

7.1 SonarCloud findings and decisions

The open-issues view identified two security issues. The CSRF warning is an intentional decision for this workshop because the gateway uses stateless HTTP Basic authentication and does not create browser sessions or cookies. The URL-construction warning was treated as a real issue and remediated by encoding gateway path variables before forwarding them to downstream services.

The screenshot displays the SonarCloud interface for the 'Digi Bank Microservices' project. The top navigation bar shows the project name and a dropdown menu for 'master'. Below this, there are filters and sorting options. The 'Filters' section shows 'Status: Open' and 'Clear filters'. The 'Sort by Priority' dropdown is set to 'Priority'. The main content area lists issues with their details, including the file path, issue description, severity, and effort.

File Path	Issue Description	Severity	Category	Effort	Age
api-gateway/.../main/java/com/digibank/gateway/SecurityConfig.java	Make sure disabling Spring Security's CSRF protection is safe here.	High	Security	5min effort	2 hours ago
api-gateway/.../main/java/com/digibank/gateway/GatewayController.java	Change this code to not construct the URL's path from user-controlled data.	Medium	Security	30min effort	12 hours ago
account-service/.../com/digibank/account/error/ApiError.java	Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method.	Info	Maintainability	5min effort	12 hours ago

Informational timestamp findings using `LocalDateTime.now()` are recorded as a maintainability follow-up. They do not represent an authorization, injection or secret-exposure defect.

7.2 JaCoCo coverage evidence

JaCoCo records which production lines are exercised by the automated tests. The report is generated during Maven verify and is imported by SonarCloud. Coverage is evidence about test reach; it is not the same as proving that the tests detect incorrect behavior.

```
➔ digibank-core git:(main) ✗ find digibank-microservices -path '*/target/site/jacoco/jacoco.xml'
```

digibank-microservices/compliance-service/target/site/jacoco/jacoco.xml
digibank-microservices/account-service/target/site/jacoco/jacoco.xml
digibank-microservices/api-gateway/target/site/jacoco/jacoco.xml
digibank-microservices/customer-service/target/site/jacoco/jacoco.xml
digibank-microservices/transaction-service/target/site/jacoco/jacoco.xml
digibank-microservices/config-server/target/site/jacoco/jacoco.xml
digibank-microservices/discovery-server/target/site/jacoco/jacoco.xml



api-gateway

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.gateway		57%		25%	20	42	30	80	13	34	0	6
Total	198 of 461	57%	12 of 16	25%	20	42	30	80	13	34	0	6



transaction-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.transaction.service		17%		0%	14	17	16	19	11	14	1	2
com.digibank.transaction.client		0%	n/a	n/a	9	9	11	11	9	9	4	4
com.digibank.transaction.error		0%	n/a	n/a	9	9	9	9	9	9	2	2
com.digibank.transaction.controller		0%	n/a	n/a	6	6	6	6	6	6	1	1
com.digibank.transaction.dto		61%	n/a	n/a	1	2	1	2	1	2	1	2
com.digibank.transaction.pattern		83%		75%	2	9	5	21	0	5	0	2
com.digibank.transaction		0%	n/a	n/a	2	2	3	3	2	2	1	1
com.digibank.transaction.model		93%	n/a	n/a	1	9	1	3	1	9	0	1
Total	419 of 582	28%	8 of 14	42%	44	63	52	74	39	56	10	15

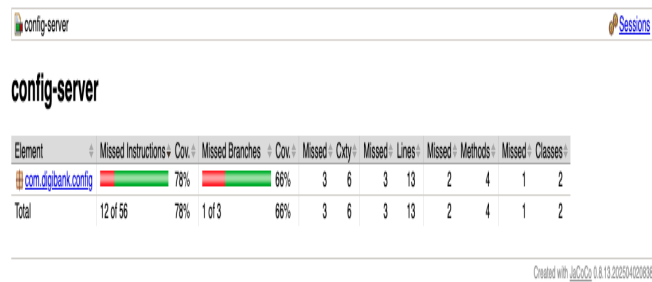
Created with JaCoCo 0.8.13.202504020838



compliance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.compliance.service		8%		0%	10	12	7	9	9	11	0	1
com.digibank.compliance.error		0%	n/a	n/a	9	9	9	9	9	9	2	2
com.digibank.compliance.model		0%	n/a	n/a	10	10	4	4	10	10	1	1
com.digibank.compliance.dto		0%	n/a	n/a	3	3	3	3	3	3	3	3
com.digibank.compliance.controller		0%	n/a	n/a	7	7	7	7	7	7	1	1
com.digibank.compliance		0%	n/a	n/a	2	2	3	3	2	2	1	1
Total	344 of 355	3%	2 of 2	0%	41	43	33	35	40	42	8	9

Created with JaCoCo 0.8.13.202504020838



Coverage reports are available for the tested services. The notification service has no test sources and therefore produces no execution data.

7.3 PITest mutation testing

PITest changes compiled bytecode in controlled ways, such as negating a condition or replacing a return value. A killed mutant means a test detected the change. A surviving or uncovered mutant identifies a branch where the current tests are not strong enough to detect that behavior change.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
6	63% 50/80	12% 4/34	24% 4/17

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.gateway	6	63% 50/80	12% 4/34	24% 4/17

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Enhanced functionality available at [arcmutate.com](#)

The gateway report generated 34 mutations, killed 4, and recorded 24% test strength. This is a test-strength observation, not a failed scan.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
5	27% 12/44	10% 3/29	50% 3/6

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.customer.controller	1	0% 0/6	0% 0/5	100% 0/0
com.digibank.customer.error	2	0% 0/9	0% 0/7	100% 0/0
com.digibank.customer.model	1	64% 9/14	25% 1/4	25% 1/4
com.digibank.customer.service	1	20% 3/15	15% 2/13	100% 2/2

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100% 9/9	100% 3/3	100% 3/3

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.discovery 1	1	100% 9/9	100% 3/3	100% 3/3

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Enhanced functionality available at [arcmutate.com](#)

The other service reports show that mutation results are module-specific; they should be reviewed as test-improvement input rather than combined into one misleading score.

7.4 Static-analysis delivery controls

Static checks are exposed through simple Make commands so the instructor and the team can reproduce the same controls locally.

```
make build
make sast-sonar
make sast-pitest
make sast-dependency-check
```

The CI workflow runs SonarCloud as a separate job, runs Dependency-Check and PITest in the static-analysis job, uploads reports even when a scan fails, and caches the Dependency-Check NVD database. The first NVD database download is large; later runs reuse the GitHub Actions cache and retrieve only updates.

Control	Evidence and decision
SonarCloud	Quality gate passed; issues reviewed and classified.
JaCoCo	Coverage generated and imported into SonarCloud.
PITest	Mutation reports generated; surviving mutants become test-strength follow-up.
Dependency-Check	NVD API key and cache configured; final report remains pending until the first database update completes.
CI	Separate jobs, secret-based credentials, report upload and cache reuse.

8. Deliverable mapping

Objective	Demonstrated by
A2.1-A2.2	SAST/Shift Left explanation and source-security review scope.
A2.3	Review of validation, information exposure, dependency and test-strength risks.
A2.4	Maven, SonarCloud, JaCoCo, PITest, Dependency-Check and GitHub Actions.
A2.5	SonarCloud findings interpreted as remediation, accepted decision or follow-up.
A2.6-A2.7	Gateway path hardening, PITest configuration fix and documented security decisions.
A2.8	Passing build/tests, runtime DAST evidence and reproducible static-analysis controls.
A3.1-A3.8	Workshop 1 Newman/ZAP report and runtime evidence retained in this continuation PDF.

The result is a documented chain from source and dependency analysis, to risk interpretation, to remediation, to runtime revalidation.